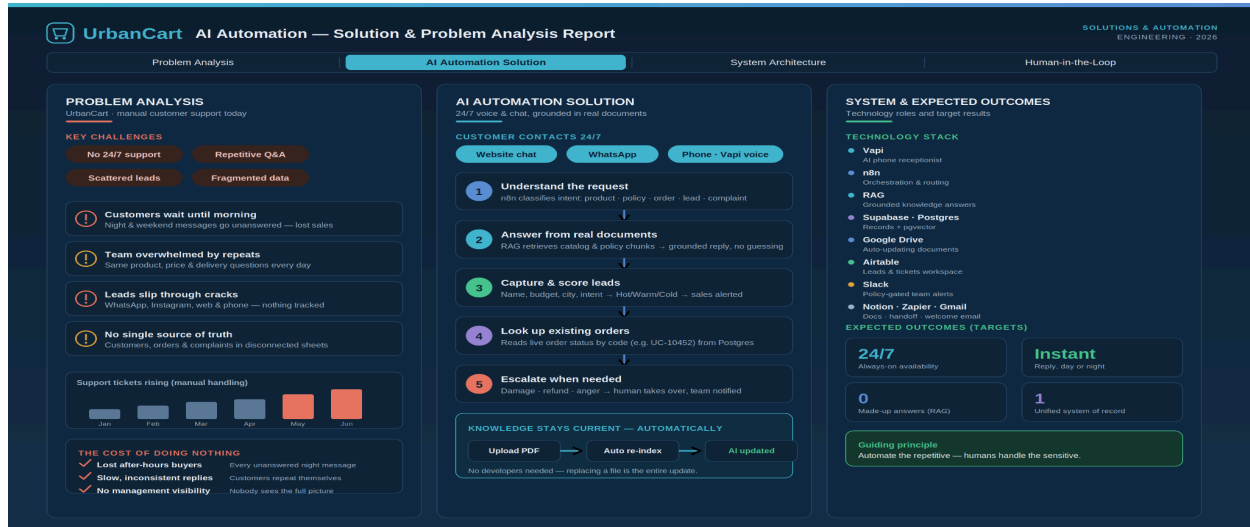


UrbanCart

AI Customer Experience & Operations Automation

Task 1: Solution Analysis & Proposal



Prepared By: Anila Gulzar Toor
(Solutions & Automation Engineer)

Project: MATalogics Internship Final Task

Client: UrbanCart

Industry: E-Commerce, Pakistan

Based on: Discovery Meeting Minutes

Attendees: 1. Ibrahim Wasiq Qurashi (Business Developer)
2. Sarah Malik (UrbanCart Operations Manager)

Held on: 1 September, 2026

Submitted To: Muhammad Ahmad Tahir
(Founder and CEO MATalogics)

Table of Contents

SCOPE OF DOCUMENT	2
1. Business Problem Analysis	2
Success criteria stated by the client:	2
2. Automation Boundary — What Is Automated vs. Human-Controlled	3
• Fully automated (AI, 24/7):	3
• Automated with human notification:	3
• Human-controlled (AI hands off immediately):	3
3. Users & Customer Journeys	3
External users:	3
Internal users:	4
Journey A — Product/policy question (chat, 2 a.m.)	4
Journey B — Voice caller with purchase intent	4
Journey C — Order status	4
Journey D — Complaint requiring a human	4
Journey E — Knowledge update (internal)	4
4. System Architecture	4
Technology Roles & Justification (as required)	5
5. Data Model (PostgreSQL in Supabase)	7
Database Tables	7
6. The RAG Pipeline (ingestion → grounded answers → updates)	8
Document Ingestion	8
Text Extraction	9
Chunking	9
Embeddings	9
Vector Search & Retrieval	9
Grounded Responses	9
The Honesty Rule	9
Knowledge Updates	9
7. Voice & Chat Interaction Design	10
Chat (website widget + WhatsApp)	10
Voice (Vapi)	10
8. Lead Capture, Scoring & Sales Handoff	10
9. Escalation Order Retrieval	11
10. Human Escalation Design	11
Escalation triggers:	11
On trigger:	11
11. Internal Team Notification Design	11
12. Error & Missing-Information Handling	11
13. Non-Functional Priorities Alignment	12
14. Conclusion	12

SCOPE OF DOCUMENT

This document is the Task 1 deliverable, a Solutions Architect's analysis of UrbanCart's discovery meeting and the resulting solution design, what problems exist, what should be automated versus kept under human control, who the users are, what their journeys look like, and how the system as a whole (architecture, data, RAG, voice/chat, escalation, notifications) is designed to work. It does not cover build steps, workflow configuration, or platform setup which is the subject of the companion document "Task 2: Implementation Plan".

1. BUSINESS PROBLEM ANALYSIS

From the meeting minutes, UrbanCart's problems fall into five distinct categories:

S/N	Problem (in the client's words)	Underlying issue
1	"Our biggest problem is customer support... answered manually."	High-volume repetitive Q&A consuming the team
2	"If someone sends a message at night, they wait until the next day. We're losing customers."	No 24/7 coverage → lost revenue
3	"Sales receives leads through different places; it's difficult to track."	No unified lead capture or pipeline
4	"Data in different places... spreadsheets, e-commerce system, another sheet for complaints."	Fragmented data, no system of record
5	"Someone has to manually update everything... we don't want developers changing the system every time we update a PDF."	Knowledge maintenance requires technical work

Success criteria stated by the client:

- 24/7 chat + voice availability
- answers grounded in *actual* business documents (no hallucination)
- qualified leads routed to sales
- basic order inquiries automated
- immediate human escalation for serious issues
- low-noise notifications, and management visibility.

The client explicitly accepts a phased rollout ("start with the most important functionality and expand later") within **6–8 weeks**.

2. AUTOMATION BOUNDARY (Automated vs. Human-Controlled)

The client was explicit: *“The AI should not try to solve everything.”* The design principle is to **automate the repetitive and escalate the sensitive**:

- **Fully automated (AI, 24/7):**
 - Product availability, price, specs questions (from live product data)
 - Policy questions, delivery time/areas, returns, warranty (via RAG over UrbanCart’s real documents)
 - Order-status lookups by order code (e.g., “UC-10452”)
 - Lead capture: name, phone, product interest, budget, location, purchase intent
 - Conversation logging and routine records, no notifications for normal questions
- **Automated with human notification:**
 - High-value lead detected → Slack alert to sales with full context
 - Failed automation/system error → Slack alert to ops
- **Human-controlled (AI hands off immediately):**
 - Damaged product reports, complicated refunds, angry/frustrated customers → ticket created, support alerted as high-priority, AI stops attempting resolution
 - Any question where retrieved knowledge is insufficient → AI says a human will assist rather than guessing (explicit client requirement)
 - Final say on knowledge content: staff control which documents are published to the AI

3. USERS & CUSTOMER JOURNEYS

External users:

UrbanCart customers on the

- ❖ website (chat widget)
- ❖ WhatsApp, and
- ❖ Phone.

Internal users:

- ❖ Sales (leads)
- ❖ Support (tickets & orders)
- ❖ Operations (tasks/issues)
- ❖ Management (overview)

Journey A — Product/policy question (chat, 2 a.m.)

Customer asks “Can I return headphones after 10 days?” → chat webhook → n8n → RAG retrieval over the *actual* return policy → grounded answer with the specific policy terms → conversation logged. No humans involved, no notification.

Journey B — Voice caller with purchase intent

Customer calls → Vapi AI receptionist greets, understands intent → answers product questions (via the same RAG/tools) → detects buying interest → conversationally collects **name, phone, product, budget, location, ready-to-buy vs. browsing** → lead saved → sales sees a Slack message: “*New customer interested in iPhone 15, budget Rs. 200,000, located in Lahore*” with a link to the full record — including any **previous history**, so the team never starts from zero (explicit requirement).

Journey C — Order status

“My order UC-10452 hasn’t arrived.” → AI extracts the order code → queries the orders database → responds with live status and expected delivery → logged. If the order is flagged problematic (e.g., long-delayed), an important order issue alert goes to support.

Journey D — Complaint requiring a human

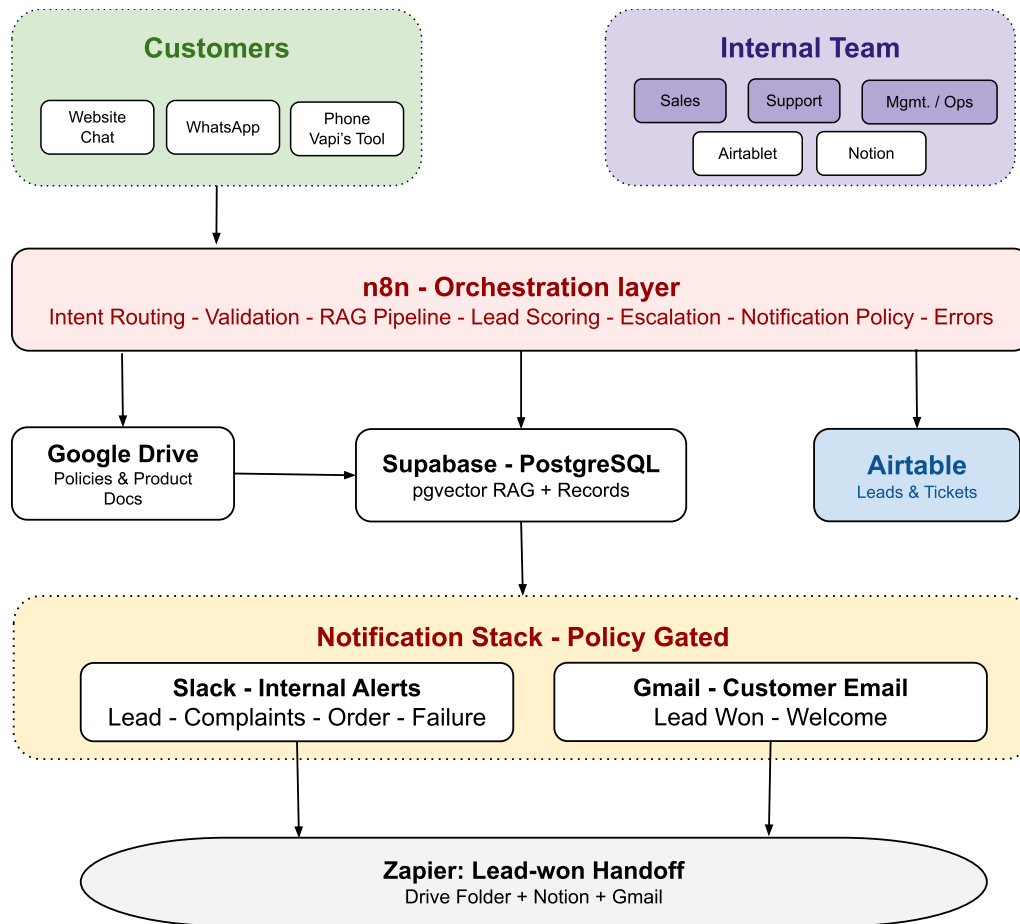
“My product arrived damaged and I want a refund now.” → intent + sentiment detection classifies this as human-required → AI responds empathetically that a support member is taking over → high-priority ticket created with the transcript → immediate Slack alert to support. AI does not attempt resolution.

Journey E — Knowledge update (internal)

Ops uploads a new Shipping Policy v3.pdf to the designated Google Drive folder → the RAG pipeline automatically re-ingests it → old version’s chunks retired → AI answers reflect the new policy. No developer involvement (explicit requirement).

4. SYSTEM ARCHITECTURE

At a high level, customers reach the system through three channels (website chat, WhatsApp, phone). All channels feed a single orchestration layer in n8n, which performs intent routing, validation, RAG retrieval, lead scoring, escalation decisions, errors and the notification policy. n8n reads and writes the system of record (Supabase/PostgreSQL), the team-facing pipelines (Airtable), the knowledge source (Google Drive → pgvector), and pushes outputs to the internal team (Slack, Notion) and, for the sales handoff, to Zapier.



Technology Roles & Justification (as required)

Technology	Role	Why Appropriate
Vapi	The AI phone receptionist: answers calls 24/7, understands intent, calls n8n tool-webhooks for knowledge / orders / leads, escalates	Purpose-built voice AI, client explicitly wants phone coverage and lead capture by voice
n8n	Central orchestrator: all routing, the RAG ingestion pipeline, validation, escalation logic, error handling	One controllable hub for every flow; self-hostable = reasonable cost (client priority)
Zapier	One discrete business workflow: when sales marks a lead Won in Airtable → create Google Drive deal folder + Notion handoff page + Gmail welcome to the customer	Demonstrates cross-app business automation on the <i>sales</i> side without entangling the core support orchestration; easy for non-technical staff to maintain

Technology	Role	Why Appropriate
Google Drive	The document source of truth: staff drop/replace policy & catalog files (PDF/Word/Sheets) in one folder	Client already maintains documents as files; Drive gives them a zero-training update path that triggers automatic re-indexing
Airtable	The human working layer : Leads pipeline (sales) and Support tickets (support) as views/kanban	Non-technical teams need a friendly daily workspace; distinct data from Postgres, so no redundant duplication
Notion	Internal documentation & knowledge management: SOPs, escalation playbook, document registry, project docs	Client <i>already uses Notion</i> and asked for knowledge/project info there
Slack	Internal notifications under a strict policy: high-value lead, serious complaint, failed automation, important order issue — nothing else	Client uses Slack and explicitly demanded low noise
PostgreSQL	The relational engine (running inside Supabase): canonical records; customers, orders, conversations, leads history, documents	Orders/customers are relational data needing integrity and joins (“see the customer’s previous information”)
Supabase	The backend platform providing that PostgreSQL plus pgvector (embeddings), REST APIs, and auth	One platform covers structured data <i>and</i> vector search — fewer moving parts = reliability, easy maintenance, reasonable cost (all stated client priorities)
RAG	Grounded answering over UrbanCart’s real product / shipping / return / warranty / support documents	Client’s hardest requirement: “don’t make up information... use our actual information,” with automatic knowledge updates

Why Supabase + PostgreSQL are one deliberate choice, not two:

Supabase is the managed platform; PostgreSQL is the database engine inside it.

Using Supabase gives UrbanCart production Postgres for relational data *and* pgvector for RAG in a single, low-maintenance service, directly serving their reliability, maintenance, and cost priorities.

Listing them separately reflects two distinct *roles* (relational store; backend/vector platform), not two systems.

Why both Postgres and Airtable without redundancy:

Postgres is the canonical system of recording the automation reads/writes; Airtable holds only the two team-facing pipelines (leads, tickets) where humans drag cards and

add notes. n8n syncs status changes back. Each store answers a different question: “what is true?” (Postgres) vs. “what am I working on today?” (Airtable).

5. DATA MODEL (PostgreSQL IN Supabase)

All operational data lives in Supabase/PostgreSQL as the single system of record, structured around eight entities. This section describes what each entity represents and why; the concrete schema is detailed in the Task 2 Implementation Plan.

Database Tables

The eight tables:

1. **Customers:** One row per person who has ever contacted UrbanCart. Stores their
 - Name
 - Phone number
 - Email
 - City, and
 - which channel they first came through (website, WhatsApp, or a phone call)

The **phone number is the key**: whether someone chats today and calls tomorrow, the system recognizes them as the same person, which is how the sales team sees a customer's history "instead of starting from zero."

2. **Leads:** One row per potential buyer. Records
 - what product they're interested in
 - their approximate budget (in PKR)
 - their city
 - whether they're ready to purchase or just browsing, and
 - the AI's score (Hot, Warm, or Cold)

Each lead is linked back to a customer record, so a lead is never an anonymous entry — it's always attached to a real, known person.

3. **Orders:** One row per purchase. Holds the
 - order code the customer quotes on the phone (like "UC-10452")
 - what was ordered
 - the total amount
 - when it was placed
 - its current status (placed → packed → shipped → delivered, or delayed), and
 - the expected delivery date

This is the table the AI reads when someone asks "where is my order?" — so the answer is always the *live* status, never a guess.

4. **conversations:** A diary of every interaction. Each row records
 - which customer it was
 - which channel (chat, WhatsApp, or voice)

- what was said
- what the customer wanted, and
- how it ended (answered automatically, became a lead, or was escalated to a human)

This is what gives management "a clear view of what's happening" and gives the team full context when they take over.

- 5. Tickets:** One row per problem that needs a human: a damaged product, a complicated refund, an angry customer. It notes the category, the priority, which customer and (if relevant) which order it concerns, and links to the matching card in Airtable where the support team actually works on it.
- 6. Documents:** The catalog of what the AI is allowed to know. One row per business document (product catalog, return policy, shipping policy, warranty info, support guidelines) with its version number and whether it's the *active* version or a *retired* old one. When staff upload a new policy PDF, the old version is retired here, so there's always a clear record of exactly which policy the AI is answering from.
- 7. Doc_chunks:** The documents above, cut into small searchable pieces. Each piece keeps its text plus a mathematical "fingerprint" (the embedding) that lets the AI instantly find the most relevant pieces when a customer asks a question. This table is the AI's actual memory — it can only answer from what's stored here, which is what prevents made-up answers.
- 8. Events:** A simple, append-only logbook of everything the system does: lead created, document re-indexed, conversation escalated, automation failed. Nothing is ever edited or deleted here, only added, so if anything ever needs investigating, there's a complete trail of what happened and when.

6. THE RAG PIPELINE (ingestion → grounded answers → updates)

The client's hardest requirement is explicit: "We don't want the AI making up information... it should use our actual information." Retrieval-Augmented Generation (RAG) is the mechanism that grounds every product/policy answer in UrbanCart's real documents rather than the model's general knowledge.

Document Ingestion

Staff drop or replace files (product catalog, return policy, shipping policy, warranty info, support guidelines, internal training docs, PDF, Word, or Excel) in a designated Google Drive folder. This is the entire update procedure from the client's side; no developer touches the system.

Text Extraction

PDFs and Word documents are converted to plain text; spreadsheets (e.g. the product catalog) are converted row-by-row into structured statements ("Product: X |

Price: Y | Stock: Z | Warranty: W”) so tabular facts remain retrievable as discrete statements.

Chunking

Extracted text is split into overlapping ~300–500 token chunks. Policy documents are chunked along section headings so a single chunk carries one complete rule (e.g. the full return window clause), avoiding answers built from a fragment of a rule.

Embeddings

Each chunk is converted into a vector embedding that captures its meaning, allowing search by semantic similarity rather than exact keyword match.

Vector Search & Retrieval

A customer question is embedded the same way and compared against the active chunks using vector similarity search (top-k), optionally filtered by document type when intent is already known (e.g. a returns question searches the return policy first).

Grounded Responses

Retrieved chunks are injected into the AI's prompt with an explicit instruction: answer only from the provided context. The response quotes or paraphrases the retrieved policy/product text rather than generating from general knowledge, and the sources used are logged with the conversation for auditability.

The Honesty Rule

If retrieval confidence is low, or the retrieved context does not contain the answer, the AI does not guess. It tells the customer that a team member will follow up and — for anything with potential business impact — opens a ticket. This is enforced both in the prompt and by a minimum retrieval-score threshold.

Knowledge Updates

Replacing a file in Drive is the entire update procedure. When a new version of a document is uploaded, its old chunks are marked retired and new chunks become active automatically, typically within minutes. A staff-facing Notion “Knowledge Registry” page shows exactly which documents and versions the AI currently knows, satisfying the client's requirement that non-developers can see and trust what the AI knows.

7. VOICE & CHAT INTERACTION DESIGN

Chat (website widget + WhatsApp)

Messages hit an n8n webhook → intent classification (product | policy | order_status | lead | complaint | other) → routed to the matching sub-flow (RAG answer, order lookup, lead capture, or escalation) → reply returned to the channel. WhatsApp and the website share the same brain; only the in/out connectors differ, so behavior is consistent across channels.

Voice (Vapi)

The AI receptionist is configured with a system prompt (UrbanCart persona, honesty rule, escalation rules) and set of callable **tools** that invoke n8n webhooks:

- ★ search_knowledge(question) → RAG retrieval → grounded spoken answer
- ★ lookup_order(order_code) → Postgres query → status read back naturally
- ★ capture_lead(name, phone, product, budget, city, intent) → validates & saves → triggers scoring
- ★ escalate(reason, summary) → ticket + Slack, then a graceful handoff message

Vapi handles the conversational voice layer (speech recognition, turn-taking, barge-in, text-to-speech); all business logic stays in n8n, so voice and chat share a single implementation of the truth. The AI never behaves differently depending on channel.

8. LEAD CAPTURE, SCORING & SALES HANDOFF

Captured fields (exactly as the client listed): name, phone, product interest, approximate budget, location, ready-to-purchase vs. just asking.

Scoring runs in n8n on save — budget and intent carry the most weight:

Hot = ready-to-purchase + budget within range of the product's price → *immediate* Slack alert to #sales with the client's example format: "New customer interested in iPhone 15, budget Rs. 200,000, located in Lahore" + link to the Airtable record (which shows prior conversations/orders via the phone-number identity).

Warm = genuine interest, browsing or unclear budget → appears in the Airtable pipeline, **no** Slack ping (noise policy).

Cold = casual inquiry → logged only.

When sales marks a lead **Won** in Airtable, the **Zapier** workflow fires: Drive deal folder + Notion handoff page + Gmail welcome, the sales-side business automation, cleanly separated from support orchestration.

Customer issues (complaints) are captured the same way conversations are logged, but are additionally classified by severity, see Sections 10 - 11 for escalation and notification handling.

9. ESCALATION ORDER RETRIEVAL

When a customer states or is asked for an order code (e.g. “My order UC-10452 hasn't arrived yet”), the AI extracts the code from the utterance/message, queries the Orders table directly, and reads back the live status and expected delivery date — never a cached or guessed value. If the order is flagged as delayed or otherwise problematic, an “important order issue” notification is raised to support (see Section 11). If no matching code is found, the AI asks the customer to confirm the code once before falling back to human follow-up (see Section 12).

10. HUMAN ESCALATION DESIGN

Escalation triggers:

A damaged product report, a refund request beyond a simple policy answer, detected anger or frustration, an explicit “I want to talk to a human,” or low-confidence retrieval on an important issue.

On trigger:

The AI sends an empathetic handoff message, creates a ticket row in Postgres and mirrors it as a card in Airtable, and sends an immediate Slack alert to #support-urgent with a transcript summary and a link to the customer's history. The AI does not attempt to resolve the issue itself once it is classified as human-required.

11. INTERNAL TEAM NOTIFICATION DESIGN

The client's exact list is enforced by a single “should-notify” gate in the orchestration layer: only a high-value lead, a serious complaint, a failed automation, or an important order issue generates a Slack notification. Everything else — normal questions, warm/cold leads, routine order lookups — is silent in Slack but fully visible in Airtable and Notion, matching the client's explicit request to keep Slack from becoming noisy.

12. ERROR & MISSING-INFORMATION HANDLING

- Input validation at every webhook: malformed order codes trigger a polite re-ask; an invalid phone number during lead capture triggers a re-prompt.
- Order code not found: the AI asks the customer to confirm the code once, then offers human follow-up and logs a ticket — it never invents a status.
- Missing/insufficient knowledge: per the honesty rule (Section 6), the AI tells the customer a team member will assist rather than guessing.

- Automation failures: every workflow has retry logic (with backoff) for transient failures; on final failure, a Slack alert goes to #ops-alerts (one of the client's four notification categories) and the failure is logged for traceability.
- Voice call failures: the assistant offers a callback and logs the number as an urgent follow-up task.
- Management visibility: a silent daily digest (conversations handled, % answered by AI, escalations, failures) gives management “a clear view of what's happening” without adding Slack noise.

13. NON-FUNCTIONAL PRIORITIES ALIGNMENT

Priority	How Design Addresses
Reliability	Single orchestration hub (n8n) with a global error workflow and retries; Supabase-managed Postgres with backups
Security	All integrations via authenticated webhooks/API keys stored as credentials, not in workflow logic; role-based access in Airtable/Notion/Slack
Easy Maintenance	Knowledge updates require only a file replacement in Drive; Airtable/Notion give non-technical staff a friendly daily workspace
Reasonable Cost	n8n is self-hostable; Supabase bundles Postgres + vector search in one managed service, avoiding duplicate infrastructure
Ability to Scale	Stateless webhook-driven workflows and a managed Postgres backend scale independently of any single channel
Accurate AI Responses	RAG grounding plus the explicit honesty rule and a retrieval-confidence threshold prevent fabricated answers

14. CONCLUSION

This design gives UrbanCart 24/7 grounded chat and voice coverage, a single system of record for customers/leads/orders/tickets, automatic knowledge updates with no developer involvement, and a notification policy that keeps the internal team informed without noise, while keeping every sensitive interaction (damaged goods, refunds, angry customers) in human hands. The companion document, Task 2: Implementation Plan, translates this design into a concrete, platform-by-platform build plan and a phased 6-8 week delivery schedule.

